Course Scheduling Platform

Spark Team | Spring, 2026

Elianna Bottone, Fatma Toprak, Matthew Kaminski, and Michael Kudrik

1. Abstract

This report details the design and implementation of a web-based course scheduling system developed for COSMC 330 at Roger Williams University. The application enables professors to create events, share them with prospective students, and view the aggregated availability. The system was built as a full stack web application and covers the complete software development lifecycle from requirements through deployment.

2. Executive Summary

This report covers the Class Planner application built by Spark Team (Team 5) for COMSC 330 at Roger Williams University during the 2026 spring semester. The goal for the application was to allow a professor to create a special topic's event, and then share it with students who are interested. The students are then able to choose which days on a given week they are available to attend. Finally, the professor who created the event will be able to navigate to the results page, which details what days had the most students available.

The system was built as a full stack application, allowing the responses to be saved to a centralized database, and displayed within the frontend. The user interface was built using the JavaScript framework React, while the backend was constructed using a python library called fastAPI. The REST endpoints communicated with our database that uses MariaDB. Our team also chose to use Docker for its helpful Docker Compose feature that allowed us to containerize the database and allow for easy development across the team.

Throughout this report we detail the full software development lifecycle, such as requirements, system design, implementation details, testing procedures, and our deployment considerations. Key design decisions such as our relational database schema, user experience choices, and development environments are documented.

Contents

1. Abstract 1
2. Executive Summary 1
3. Abbreviations and Glossary 3
4. Introduction 4
5. Requirements 4
6. Design & Implementation 4
   6.1. How the Website Works 5
7. Development Environment 5
8. Target Environment 6
9. Testing 6
10. Deployment 6
11. Summation and Conclusions 7
12. Acknowledgements 7
13. References 7
14. Appendices 7
   14.1. Design Deliverables 7
   14.2. User Manuals 8
      14.2.1. Professor Guide 8
      14.2.2. Student Guide 9
      14.2.3. Development Guide 10
      14.2.4. Deployment Guide 10
         14.2.4.1. Services 10
         14.2.4.2. Dockerfiles & Internals 10
         14.2.4.3. Prerequisites & Runing the Application 11
   14.3. Source Code 11

3. Abbreviations and Glossary

API Application Programming Interface
Backend The "server-side" of applications.
Frontend The user-facing part of a computer system.
GET request HTTP method used to retrieve or read data from a web server.
Git Used to track changes in source code during software development.
JSON JavaScript Object Notation.
PIP Standard package manager for Python.
PNPM Performant NPM.
POST request An HTTP method used to send data to a server to create or update a resource.
RESTful API Representational State Transfer API.
RWU Roger Williams University.
Tmux Terminal multiplexer for Unix-like systems.
URL Uniform Resource Locator.
Vim A text editor.

4. Introduction

The idea behind this project stemmed from the need to easily identify how many students are interested in a course, and when they are available to fit it into their schedules. While pre-existing solutions do exist, many of them require complex registration steps or offer far more functionality than is necessary for our shareholders use case.

To address this need, we developed a lightweight scheduling tool that is tailored specifically for Roger Williams University, yet easily adapted to other course formats. The system displays the standard RWU time blocks, allowing a professor to easily select the slots they would like to offer. The professor can then take the sharable link generated by the application and distribute it to students however they see fit. The students can then simply indicate when they are available within a few clicks, and no need to create another hard to remember username and password.

The rest of this report goes through our requirements, how we designed and built the system, our testing, deployment steps, and what we learned from the whole experience.

5. Requirements

The requirements for this project were simple. Create a software that allows professors to determine the best time to host a class based on the availability of prospective students.

The software was prohibited from gathering data through external sources, such as Roger Central, the site in which students at RWU select their courses.

In addition to the core functionality, it was a requirement that the user experience be straightforward.

6. Design & Implementation

Our website is composed of three separate entities: the React frontend, the FastAPI backend, and the MariaDB database. The frontend is what is displayed to the user. It allows the professor to input their own name as well as the course name to be displayed to the students. Professors are also able to specify which days they are willing to teach the course.

After the professor submits their choices they are provided with two URLs. The first one is to be provided to the students and leads to the student view where they will fill in availability. The second link is to be bookmarked by the professor in order to view the results after students have submitted their time preferences. On the results view they see a breakdown of how many students are available for each time slot. A colored heatmap is provided reflecting those counts. Both URLs are driven by a unique hash which is generated by the backend and stored within a table in the database.

On the student side, the user is given a similar block selection component which allows them to choose the days of which they are available to take the specified course. After the student creates their schedule they click submit and are met with a submission complete

page. We specifically designed the software to check the browser's local storage so that once a user submits they are not able to submit again.

The backend is what handles RESTful APIs as well as some of the application logic. When a student submits their availability it performs a POST request, and when results need to be displayed to the professor the backend handles a GET request. This report will get into more detail regarding these endpoints later on.

For the database design we chose to use MariaDB since it is a relational database and would meet our requirements. In order to implement the database we leveraged the use of a docker-compose.yml document. This document allowed us to specify what our docker container would contain, such as MariaDB usernames, ports, and volumes.

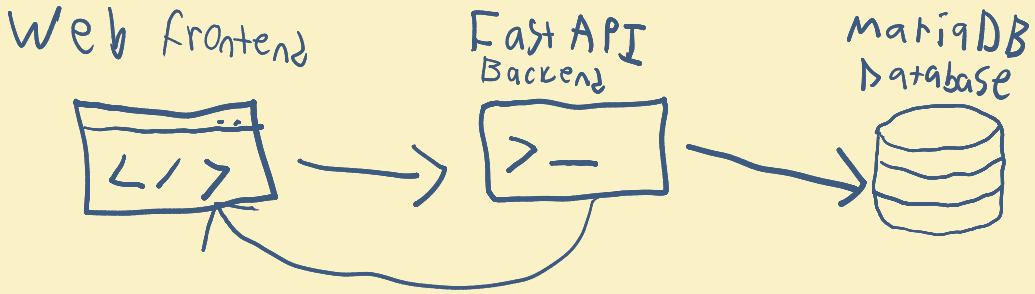

Figure 1: Application Logic

6.1. How the Website Works

- **Event Creation:** The professor creates a scheduling event through the backend. The system generates a unique hash for that event.
- **Distribution:** The professor shares the hashed URL with students.
- **Student Input:** Students visit the URL, enter their name, and click the time slots when they're free. The UI supports clicking and dragging to select multiple slots at once.
- **Data Transmission:** Student availability is sent to the backend via a POST request and stored in the database.
- **Evaluation:** The professor visits the results endpoint to see which time slots have the most students available.

7. Development Environment

During the construction of the project, there were many tools, languages, and libraries that each team member needed in order to run the project locally. To simplify the process of constructing the development environment, a document was created called overview.pdf. This document contained step-by-step instructions on installing required programs and packages, along with commands needed to run the project.

For the frontend, members were required to install the Node JavaScript runtime. To manage Node packages the program pnpm was used. On the backend, Python was required, with PIP being used for package management. The aforementioned overview.pdf listed the Python packages which required installation.

For the database, it was only necessary to have installed some version of Docker that was capable of running the Docker Compose command. By running that command, Docker created a MariaDB database locally on the computer. A seed.sql script was created that allowed Docker to insert mock tables into the database, preventing us from manually entering a schema.

During development, each member also was required to create their own .env file in order to configure needed environment variables such as the correct database credentials and ports. For version control we used Git since it allowed multiple branches to exist on the project. This meant that nobody was writing over someone else's code, and you could delegate tasks easier. The code and documentation was stored on GitHub.

As for the integrated development environments, each member of the team had their own preferences. All members shared the use of Microsoft Visual Studio Code when working on the frontend. When developing the backend and working with the database, Vim was used in conjunction with tmux.

8. Target Environment

The target environment for this project is any device with a modern web browser. Since everything client-side is browser based there is no need for additional software installation.

9. Testing

The majority of the testing was conducted manually by visiting the application's pages and verifying that they worked as intended. For the professor view, this involved creating events, visiting the generated URLs, and browsing the results. For the student view, this involved selecting availability, submitting, and ensuring that a student could not submit to the same event twice. The browser console was used to view any errors that occurred during runtime.

For the backend, the FastAPI live documentation allowed us to test the GET and POST endpoints directly. Any automated unit testing was determined to be outside the scope of this project.

10. Deployment

The system is designed for deployment either within a school's LAN, or on a cloud platform. During demonstrations, the entire application was run on a laptop running a lightweight Linux operating system and connected to the University's wireless network.

A docker compose file has been created that will allow for simple deployment. Please see Section 14.2.4 for a detailed explanation.

11. Summation and Conclusions

The team was able to successfully meet all core requirements such as class creations, availability submissions, and interest tracking. Key challenges that required overcoming were lack of knowledge on the technologies used, user interface clutter, and bridging the gap between frontend and backend data.

12. Acknowledgements

The team behind the project wants to give a thank you to all their peers for help giving feedback throughout the semester. Special thanks to Dr. Ruocco for guiding us and helping outline requirements.

13. References

- React Documentation
- FastAPI Documentation
- SQLModel Documentation
- Tailwind CSS Documentation
- Docker Compose Documentation
- MariaDB Foundation
- TanStack Router Documentation

14. Appendices

14.1. Design Deliverables

Focus Area	Deliverables & Description
Project Planning	GitHub TODO file and `overview.pdf` to streamline development environment setup.
Course Creation	Professor dashboard for inputting class titles, names, and available teaching windows.
Student Selection	A specialized view for students to express interest in specific time slots.
Submission Logic	Generation of JSON data from selected slots, transmitted to the database via REST endpoints.
Results Analytics	A teacher-facing interface displaying student availability density per time slot.

UI/UX Polish	An intuitive, easy-to-navigate interface for both user roles.
Testing & Deployment	End-to-end testing of endpoints and deployment to the university network for a live demo.

14.2. User Manuals

14.2.1. Professor Guide

1. **Access:** Navigate to the application link (e.g., `127.0.0.1:5173`).
2. **Selection:** Select the time slots during which you are willing to host the class.
3. **Details:** Enter your name and the class title.
4. **Submission:** Click the "Submit" button.
5. **Save Links:** Bookmark the generated unique links provided on the confirmation screen.
6. **Review:** Navigate to the results page via your bookmarked link to view student availability.

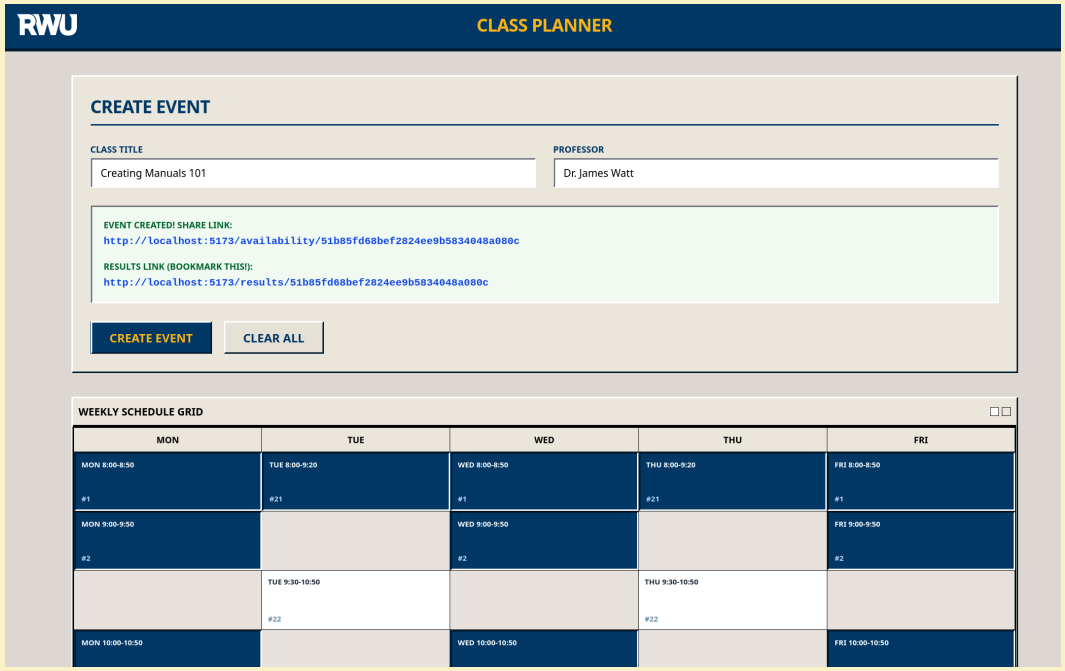

Figure 2: Professor Class Creation View

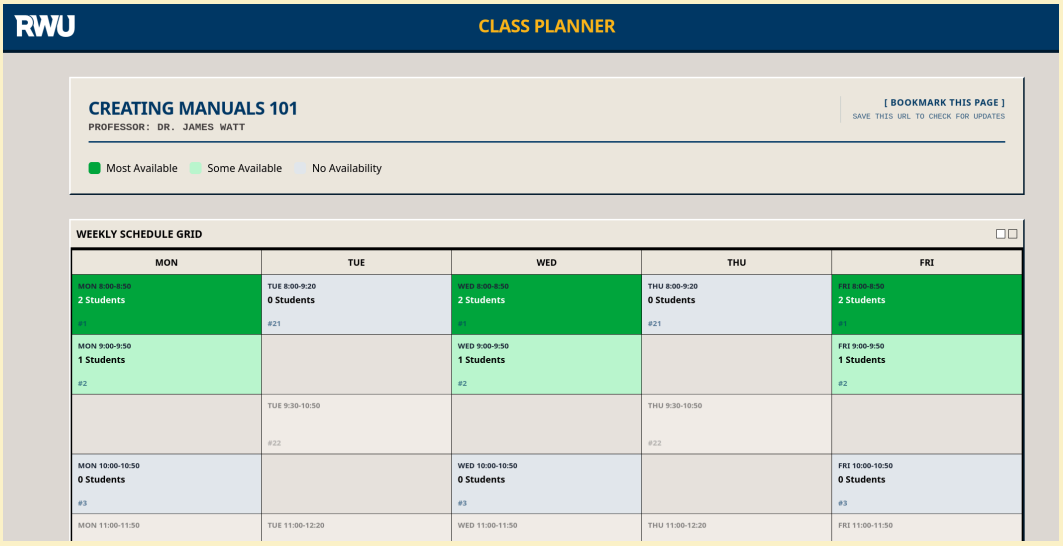

Figure 3: Professor Availability View

14.2.2. Student Guide

1. **Access:** Navigate to the specific link provided by your professor.
2. **Identification:** Enter your username and school email address.
3. **Selection:** Select the time slots during which you are available to attend class.
4. **Submission:** Click the "Submit" button to finalize your response.

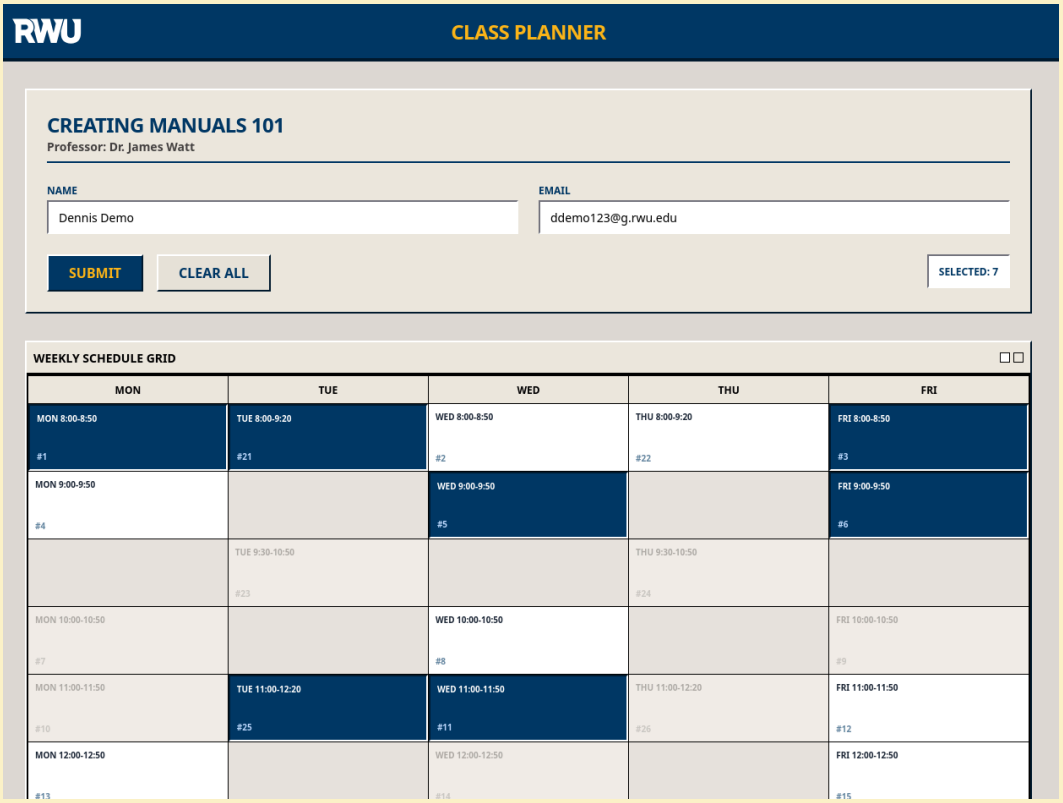

Figure 4: Student Class Selection View

14.2.3. Development Guide

For work on developing the project please view the `overview.pdf` file for instructions.

14.2.4. Deployment Guide

The application is made up of three main parts, a MariaDB database, a FastAPI backend, and the React frontend. Using Docker Compose allows users to run a single command on any device with Docker installed, and get the app running. By running this command, all three parts will be built and started in proper order, and the database will be seeded automatically.

Project Structure:

```
└─ Principles-of-Software-Design—Team-5/
   ├─ docker-compose.yml – Orchestrates all 3 services
   ├─ backend/
   │  ├─ Dockerfile – Builds the Python/FastAPI image
   │  ├─ requirements.txt – Lists Python dependencies
   │  ├─ seed.sql – Schema + dummy data, auto-run on DB init
   │  └─ main.py – FastAPI application code
   └─ frontend/
      ├─ Dockerfile – Build from Node to nginx
      └─ nginx.conf – Serves static files + proxies /api/ to backend
```

14.2.4.1. Services

The `docker-compose.yml` file has three defined services: `db`, `backend`, and `frontend`.

- The `db` service runs the `mariadb:lts` image and then ininitializes the schedulerDB database via the seed.sql script. It uses a named volume for persistant storage (meaning no data loss on container reboot), and uses a built in MariaDB healthcheck to insure the database is ready before the backend tries to connect.
- The `backend` service gets built from `./backend/Dockerfile`, and connects to the database via a `DATABASE_URL` environment variable. It starts after the healthcheck and runs on port 8000, yet routes traffic internall through the Nginx proxy.
- The `frontend` service gets built from `./frontend/Dockerfile`, and acts as the entry to our app. It runs on port 3000 and serves both the UI and the API, meaning it depends on the backend and maps external traffic to the internal container we have running on port 80.

14.2.4.2. Dockerfiles & Internals

- The `backend/Dockerfile` and `requirements.txt` use a slim python image to save storage. They install the dependencies listed in `requirements.txt` before copying the rest of the source code. Docker caches everything so dependencies won't need reinstallation after every build. The app is then started and bound to 0.0.0.0 so that it is reachable from outside the container.
- The `frontend/Dockerfile` uses a two-stage build to keep the image size down. Stage 1 uses the none:20-slim runtime and pnpm to compile the React app into a static /

dist folder of HTML, CSS, and JavaScript files. Stage 2 involves discarding everything used in step 1 except the /dist folder and the `nginx.conf` file. As a result we get a very lightweight nginx image with no overhead from development tooling.

- The `frontend/nginx.conf` file helps us configure Nginx as a reverse proxy and static host through a single port. This was done in hopes of preventing those tricky CORS issues. Both the frontend web page and the API will act as coming from the same origin.

14.2.4.3. Prerequisites & Runing the Application

Please refer to the README on the GitHub repository for detailed steps.

14.3. Source Code

Source code is available via this GitHub repository.